# Computational Vision

Laboratory Guide

Faculty: Prof. Alexandra Moutinho
Eng. Alexandre Athayde

**February 2026**

# Introduction

The present document has the objective to aid the students in the laboratory component of the curricular unit **Computational Vision** (CV), describing the objectives and required tasks to develop at each laboratory session, which aim to apply key-concepts and algorithms of Computer Vision.
Following CV's syllabus, this guide is organized into two different parts, each of them approaching separate topics of this course. The first one, **Image Processing**, presents some fundamental ideas about computational vision using a single camera and the common techniques for processing of images. This first part is then complemented by providing comprehensive insight about feature detection after the processing stage, exploring multiple techniques. In the second part, **Computer Vision**, an introduction of using deep learning for feature recognition is made, and the fundamentals of projective geometry and camera models are introduced, being used to explore the topic of stereo vision.
Each laboratory session will provide certain skills that are useful in understanding the theoretical basis of CV, and, furthermore, these are expected to help in the development of the CV projects.

- Reading materials:
  The laboratories require some level of preparation, and therefore reading materials are provided to the students using the **CV partners webpage**, and will be updated as the classes progress. These materials are the **Theoretical Classes slides** and the respective sections of the **bibliographic references**, and, in specific classes, will be complemented with other materials. Particularly for the laboratory sessions, two courses offered by MathWorks will be helpful in Part 1, **Introduction to Image Processing** (link) and **Image Segmentation, Filtering, and Region Analysis** (link), and can be audited for free from the Coursera platform.
  Since the implementation will be made using MATLAB, a set of recommended MATLAB functions (denoted in a specific font: `function`) and/or Simulink blocks (represented in italic: *Simulink Block*) is provided in the guide for each laboratory session.

- Laboratory evaluation:
  At the end of each laboratory session, a quiz will take place, with a small number of questions to be answered directly about the topics addressed in the session. These questions can be relative to the respective lab and about theoretical concepts, so adequate preparation is advised. Note that, during the implementation during the laboratory sessions, you will work in groups of 3 students, but the quiz at the end of each session is individual.

- Final Note: the recommended version of MATLAB is **2023a**, with the respective Simulink version, so please be sure to install it. If you are unable to do so, you may use the computers available in the laboratory. The required toolboxes for MATLAB are the ***Image Processing Toolbox***, ***Computer Vision Toolbox***, ***DSP System Toolbox***, ***Statistics and Machine Learning Toolbox***, and the ***Deep Learning Toolbox***. To use USB cameras, the necessary drivers for the operating system and the ***Image Acquisition Toolbox Support Package for OS Generic Video Interface*** add-on should be installed as well.

# Contents


I Image Processing 2

1 Colour Spaces and Histograms 3

2 Spatial Filters 7

3 Point Processing and Binary Morphology 11

4 Image Segmentation 16

5 Image Description Based on Keypoints 22

6 Image Description Based on Shape 29

II Computer Vision 34

7 Intelligent Vision Systems 35

8 Deep Learning for Image Recognition 39

9 Camera Calibration 45

# Part I

# Image Processing

# Chapter 1

# Colour Spaces and Histograms

Following the first-week lectures, the respective laboratory session addresses the representation of digital images, namely different colour spaces and how these are related, and provides some initial insight about the histogram of grayscale images.

## Objectives

- Familiarization with image loading and visualization tools in MATLAB and Simulink
- Experiment different colour spaces
- Learn about the histogram of a grayscale image

## MATLAB and Simulink functions

- Image loading: `imread`, *From Video Device*, *Image From File*
- Image visualization: `imshow`, `montage`, *To Video Display*
- Colour space conversion: `rgb2gray`, `rgb2hsv`, `im2gray`, *Colour Space Conversion*
- Histogram visualization: `imhist`, *2-D Histogram and Array Plot*

## Tasks

### 1. Load and visualize an image in MATLAB or Simulink

**Expected time of completion for task: 10 minutes**.

Naturally, the first step in image processing is to load an image. The image to be loaded should be a coloured one since it will be useful to explore different colour spaces. Do not forget that the image file should be in the working directory of MATLAB, or the path to it should be added.
The image used as an example is provided in the file *bottlecaps.jpg*, and is the following:

Figure 1.1: RGB example image.

### 2. Experiment using different colour spaces

**Expected time of completion for task: 20 minutes**.

After this, try converting this image into different colour spaces, namely HSV, YCbCr and grayscale. If you are using MATLAB, the `montage` command is useful to visualize multiple images. Alternatively, in Simulink, different *To Video Display* blocks can be used. The result should be similar to the following:

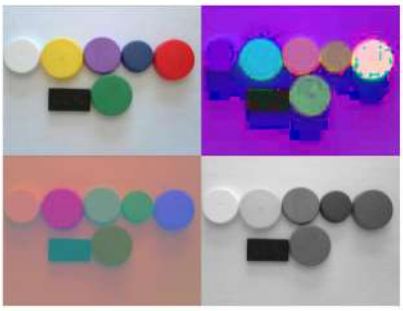

Figure 1.2: Example image in RGB (top left), HSV (top right), YCbCr (bottom left) and grayscale (bottom right).

Do not forget: you are not visualizing the images in different colour spaces, as `imshow`/`montage` interprets them in RGB; you are seeing the interpretation of RGB of a different colour space!

**3. Obtain and visualize the histogram of a grayscale image**

**Expected time of completion for task: 10 minutes**.

Lastly, obtain the histogram of a grayscale image and visualize it. As an example, a different image can be used, namely the one provided in the file *autumn.tif*, which is built-in in MATLAB (and therefore you do not need the file itself). Both the original coloured image and the respective grayscale conversion are shown on figure 1.3:

Figure 1.3: RGB and grayscale-conversion of the *autumn.tif* image (credited to MathWorks).

The respective histogram is:

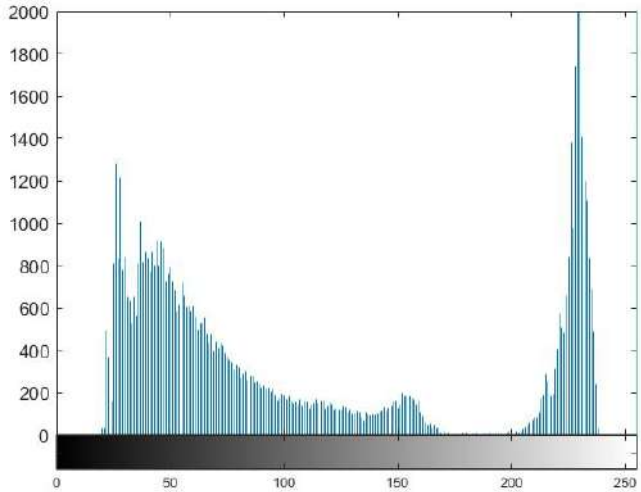


Figure 1.4: Histogram of the grayscale-conversion of the *autumn.tif* image (credited to MathWorks).

Experiment obtaining the histograms of different images and try to draw conclusions about how each of the histograms is related with the original grayscale image. What is expected to be the histogram of a dark image? Or a bright one?

**Further work: correctly select a certain colour**

Knowing how the different colour spaces work, it is now possible to filter the pixels of an image based on the values they present on a given colour space. For an 8-bit image in the RGB colour space, it can be assumed that a pixel is white if all its three components are above a certain value, which can be, for example, 200. Afterwards, it is possible to design an algorithm that generates a new image from the first one, separating the pixels that fulfil the mentioned condition and the ones that do not. Such procedure is presented on Algorithm 1, and the results shown in figure 1.5. Despite being somewhat more intricate, a similar algorithm can be used in Simulink. Please note that the separation is obviously not perfect, but this topic will be addressed in detail in further theoretical and laboratory classes.

**Algorithm 1** Detecting specific colours in an RGB image

```
[a, b, c] = size(RGB)
                                        ▷ Extracts size of the original RGB image.
NewImg = zeros(a, b)
    ▷ Generates a matrix with zeros with the same number of rows and columns of the RGB image.
for i = 1, ..., a do
    for j = 1, ..., b do
        if RGB(i, j, 1) > 200 and RGB(i, j, 2) > 200 and RGB(i, j, 3) > 200 then NewImg(i, j) = 255
                                        ▷ If all components of the pixel are above 200, it assumes it as white.

        else NewImg(i, j) = 0
                                        ▷ If not, it assumes the pixel as black.
        end if
    end for
end for
```

Figure 1.5: Detecting white-coloured pixels in figure 1.1.

Try to use the procedure described above to detect each of the colours of the bottle caps in figure 1.1. Think about which colour space is more adequate, and what conditions should be imposed in the values of the pixels. Please note that the goal here is to provide a general understanding of the colour spaces and how these can be used to extract information from images, and not to detect all the bottle caps without flaw.

# Chapter 2

# Spatial Filters

Once you know how to work with images using MATLAB and Simulink, it is now time to start to understand the more common image processing techniques. As you may know from the second lecture of this course, one of such techniques is spatial filtering, which has a multitude of different applications, all of which have their importance in image processing. In this second laboratory session, the effects of different image noise type are explored, and how such noise can be removed using spatial filters is shown.

## Objectives

- Learn how different noise types affect a grayscale image
- Explore several algorithms in spatial filtering for image smoothing
- Understand the advantages and limitations of particular algorithms

## MATLAB and Simulink functions

- Image noise: `imnoise`
- Image filtering: `fspecial`, `imfilter`, `imgaussfilt`, `medfilt2`, `conv2`, *2-D FIR Filter*
- Advanced image filters: `modefilt`, `imnlmfilt`, `imbilatfilt`, `wiener2`

## Tasks

Since Simulink itself does not have dedicated blocks for introducing noise in images nor for many of the filters to be used, a MATLAB code implementation is suggested. Nonetheless, after you complete the tasks for this session, you can, as an additional and optional task, implement the same algorithms in Simulink, for which the method is described after Task 3.
By now, you are expected to know how to load images, visualize them, and convert them to a grayscale image in MATLAB (and in Simulink), so these steps are omitted. When in doubt, refer to Laboratory 1.

### 1. Corrupt an image with different types of noise

**Expected time of completion for task: 10 minutes**.

Firstly, load one of the provided images and convert it to grayscale. Afterwards, use the aforementioned functions to introduce noise in it, and try to draw conclusions about the differences among noise types. It is useful to read the documentation of the MATLAB functions (**hint**: use the `help` command) so you know how to execute them properly, as well as which parameters they have. Experiment each noise type with different intensities as well.
As an example, we will use the *car1.jpg* image, which is shown below, already converted to an grayscale image and corrupted with different noise types:

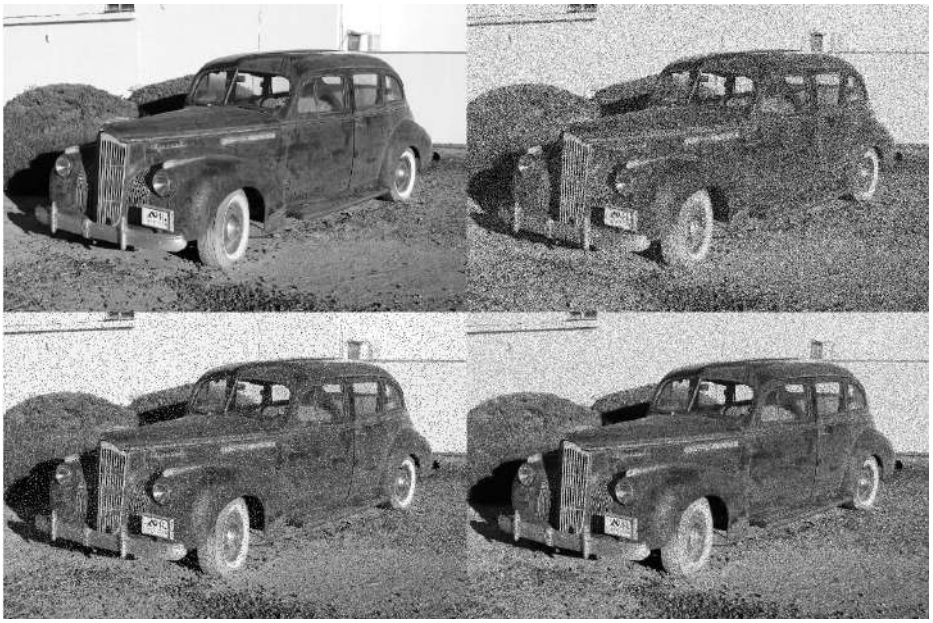

Figure 2.1: Different types of noise on the grayscale image of *car1.jpg* image (credited to MathWorks). The original image is on the top left, followed by its corruption with Gaussian (top right), salt-and-pepper (bottom left) and speckle noise (bottom right), all with 0.1 intensity.

**2. Explore different filters**

**Expected time of completion for task: 20 minutes**.

Once the image has been corrupted with noise, you can now use filters to try to suppress such noise, aiming to clean or smooth the image. Once again using the mentioned MATLAB functions, try to remove the noise of the corrupted image. For now, try doing this by applying **averaging**, a **Gaussian** and a **median** filter.
MATLAB allows you to directly filter an image by providing a respective mask, but you can achieve similar results using a convolution of such mask with the image. However, if you choose to use the latter, be aware of the data type it requires (*double*, *single*, *uint8*, ...).
An example of applying a Gaussian filter to an image corrupted with Gaussian noise is shown below:

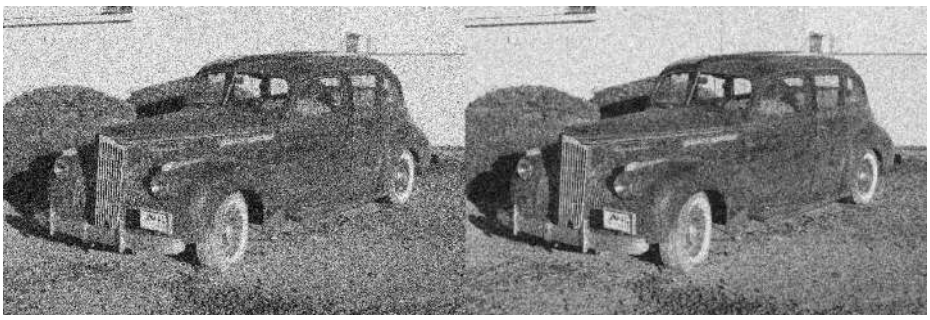

Figure 2.2: Filtering Gaussian noise by using a Gaussian filter.

Strive to determine the advantages of each filter and the respective limitations. Use several intensities for the noise, as well as different mask sizes and parameters for the filters. Does a specific filter work better for one of the noise types? Does it blur the image too much?

**3. Removing specific noise types**

**Expected time of completion for task: 20 minutes**.

Now that you have experimented with different noise types and filters, focus on removing Gaussian and salt-and-pepper noises specifically. Initially, use a relatively low intensity for the noise (for example, 0.1) and apply the same filters described in the previous task, and see how they perform when the noise increases. What is or are the best filters for each of these two types of noise?
An example of salt-and-pepper noise removal is shown on figure 2.3.

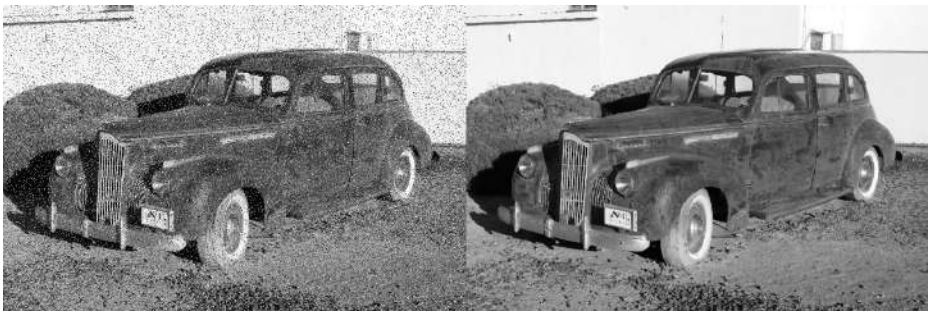

Figure 2.3: Removing salt-and-pepper noise by using an adequate filter.

Once you have experimented with the three filters and already have an idea about how these affect an image, you can work use more advanced ones like the **mode**, **non local means**, **bilateral**, **anisotropic diffusion**, or **Wiener** filters. Can you see improvements regarding the original three filters?

### Further work: Simulink implementation

Now that you completed the mandatory tasks, you may wish to implement the same noise-removal algorithm in Simulink. For this purpose, a separate Simulink model has been prepared to facilitate this implementation, and is shown in figure 2.4. In this model, the blocks inside the green and orange areas are MATLAB function blocks that allow for MATLAB functions to be called, where you can implement the noise and filtering options. Read the commentaries inside these blocks for further details. In case of any problems, call for help.

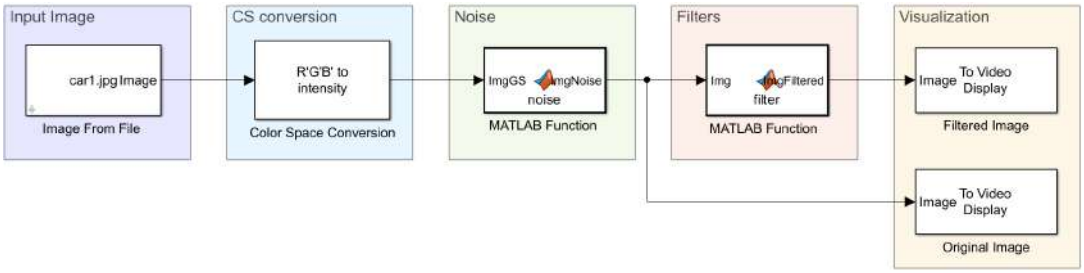


Figure 2.4: Example of a Simulink model for noise removal.

# Chapter 3

# Point Processing and Binary Morphology

Now that you have had the opportunity to work with grayscale images and learnt how to apply filtering operations for specific goals, some general tools in image processing and segmentation can be studied. In this laboratory session, the common techniques in point processing of images will be addressed, as well as the thresholding methods employed in obtaining binary images. Lastly, several morphological operations will be applied to these last images, which aim to improve the results after thresholding.

## Objectives

- Explore the common point processing techniques and study how these impact their histogram
- Learn how to threshold a grayscale image using a suitable method
- Familiarization with different morphological operations

## MATLAB and Simulink functions

- Point processing: `imadjust`, `histeq`, *Contrast Adjustment*, *Gamma Correction*, *Histogram Equalization*
- Thresholding: `graythresh`, `imbinarize`, *Compare To Constant*, *Autothreshold*
- Morphological operations: `strel`, `imdilate`, `imerode`, `imopen`, `imclose`, *Dilation*, *Erosion*, *Opening*, *Closing*

## Tasks

In the past two sessions, MATLAB code was the chosen environment for implementing the different methods and algorithms studied in the respective theoretical classes. This time however, a Simulink implementation is described in this guide, taking advantage of the fact that it provides specific blocks for the topics of this session. Nonetheless, a MATLAB code implementation is suggested as further work. Similarly to the previous laboratory class, it is assumed that you already know how to obtain grayscale images in both MATLAB and in Simulink. The image to be used is the *cells.png*, shown on figure 3.1:

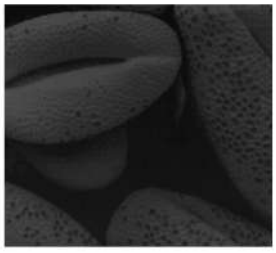

Figure 3.1: Image of *cells.png* image (credited to MathWorks).

### 1. Point Processing of Grayscale Images

**Expected time of completion for task: 20 minutes**.

In Simulink, assemble a model that allows you to visualize grayscale images using the recommended blocks, in order to be able to perform point processing operations, and to obtain the respective histogram. An example of a suitable Simulink model is provided below, where the different processing methods are implemented inside a *Subsystem* block:

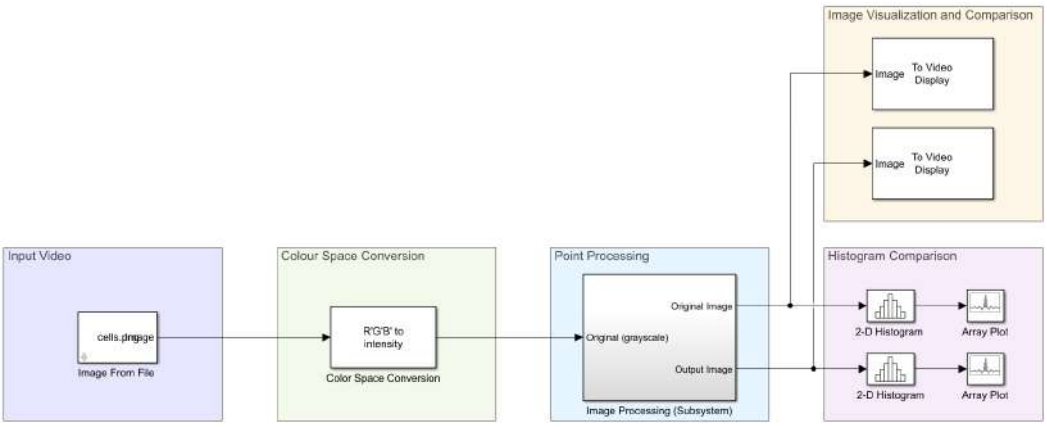


Figure 3.2: Example of a Simulink model for point processing of a grayscale image.

When using the *Image From File* block to load an image, take care to put a **discrete number as the sample time parameter** (0.1 for example), and ensure it is loaded as a **in the *double* type**, since the point processing blocks in Simulink will work for images of this type (and not *uint8*)

With the theoretical background from the third lecture, use different available blocks that allow you to perform **contrast adjustment**, **brightness adjustment**, **gamma correction** and **histogram equalization** on your grayscale image. An example of each of these operations on the original image is shown in figure 3.3, in the same order as described above.

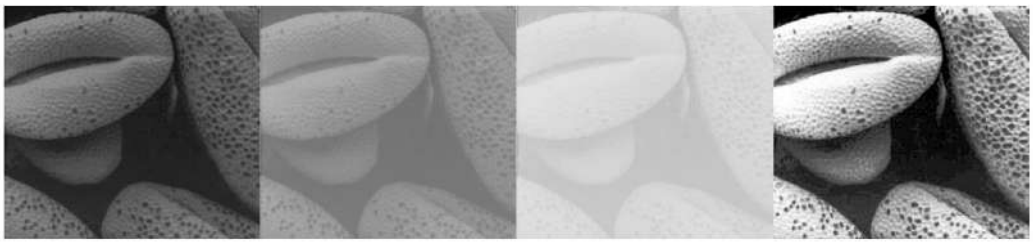

Figure 3.3: Results of the different point processing methods on a grayscale image.

See how these operations impact the respective histogram and provide an adequate explanation for the obtained results for each of them.

**2. Binarization of a Grayscale Image**

**Expected time of completion for task: 10 minutes**.

Suppose that your objective was to obtain a binary image in which the pixels that corresponded to a cell (from figure 3.1) were supposed to be white, and the remaining ones were black. Employing an adequate block, obtain a binary image using a **manual threshold**. What is the the value that provides the best results?
Afterwards, try to apply an **automatic threshold** using a suitable block. The resulting image should be similar to the one shown in figure 3.4.

Figure 3.4: Binary image after an automatic threshold method has been applied.

Is it easier to have better results using an automatic method than a manual one? Why? Could the manual method be improved if a point processing technique was used before?

### 3. Morphological Operations

**Expected time of completion for task: 20 minutes**.

In this last task, continue your process of obtaining a binary image that adequately identifies the cells in the original picture. However, you can see from figure 3.4 that, even using an automatic threshold, the obtained binary image still has multiple regions in which the pixels corresponding to a cell are not white. Apply an adequate morphological operation to your binary image and try to solve this problem, as shown in figure 3.5. Can you think of other ways, using morphological operations, of reaching a similar result? And what would be the limitations of these operations?

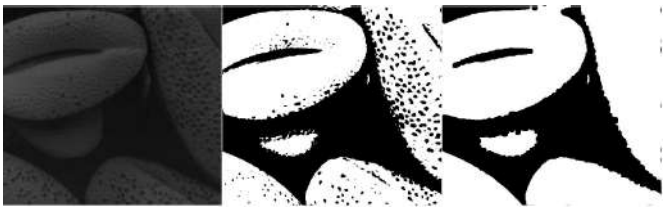

Figure 3.5: Application of morphological operations .

### Further Work: MATLAB implementation, adaptive thresholding and additional morphological operations

Regarding the MATLAB code implementation, it has the disadvantage of requiring more steps to perform the same binarization and morphological operations, but it also allows for more operations, more parameters and additional methods to be explored. Start by reading the documentation of certain MATLAB functions, namely the `imadjust`, `imbinarize` and `strel`, using the `help` command. Afterwards, repeat the same three tasks as before, but using MATLAB code instead of a Simulink model.
Lastly, add some *salt-and-pepper* noise to your grayscale image, obtain a binary version of it, and then use morphological operations to **remove the noise** that may appear in this image. Do you need to process your image before you apply the binarization method? Which technique would you use and why? To guide you in this, a suitable method of reaching the described goal of removing the noise of a binary image that adequately thresholds the pixels that correspond to cells is illustrated on 3.6. The order of operations, from the original image with noise to the final binary one, is from left to right. Can you identify each of the operations that was used?

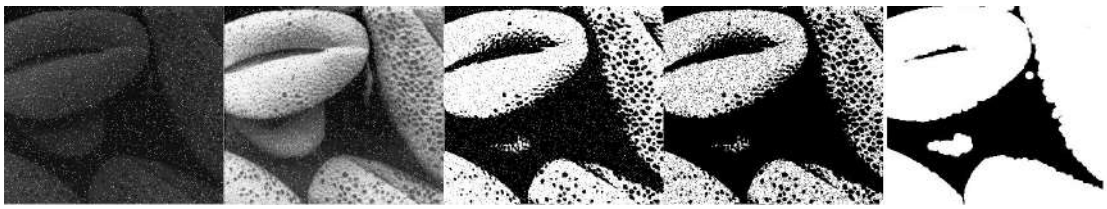

Figure 3.6: Different stages of identifying the pixels that correspond to a cell.

- Adaptive thresholding

  Now, you can progress to some more advanced topics. The first is adaptive thresholding, which is particularly useful in images with varying light conditions. For example, take the *rice.png* (credited to MathWorks) image and its binary version in figure 3.7, after using Otsu's automatic method.

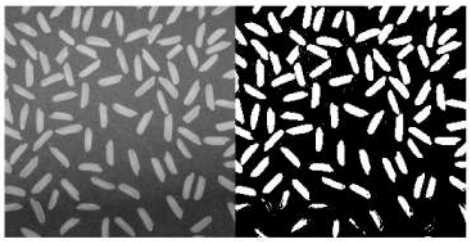

Figure 3.7: Original and binary image of *rice.png* after an automatic threshold.

As you can see, the automatic binarization method, despite providing some very good results in segmenting the rice grains in the top of the image, has a worst performance on its downside. This is due to the fact that the original image has different degrees of light, which leads to a loss of accuracy when using an automatic binarization method, since it employs a global threshold value for the image. A way to counteract this is to use an **adaptive** method that uses the mean value of the intensity of the neighbourhood of the pixel, instead of the whole image. After reading the information MATLAB provides about the `imbinarize` function, try using an adaptive binarization method, and compare it to the original image, like the figure 3.8 illustrates below:

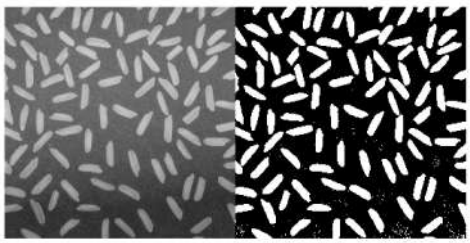

Figure 3.8: Original and binary image of *rice.png* after using an adaptive threshold.

- Additional morphological operations

  In this session, you are supposed to understand the more basic morphological operations, namely *erosion*, *dilation*, *closing* and *opening*. Explore the effect of using **different structuring elements**, with varying sizes, and how these influence the resulting binary image. Beyond these four elementary operations, there are are many other similar ones, for many of which MATLAB provides specific functions, and you can see the comprehensive list of these **here**. Experiment with them and see how they affect the resulting binary image, and try to think about scenarios in which these would be useful.

# Chapter 4

# Image Segmentation

Having concluded the first three laboratory sessions, you should now be able to perform basic image processing techniques, namely removing noise, adjusting the gray levels of the image, and obtaining a binary version of it. Binarization, in particular, requires that you define clearly what you are trying to identify - or to segment. In this laboratory, we will explore some image segmentation techniques, mainly how to detect edges, corners and circles, and how spatial filtering can be used to enhance such detection. As an additional and optional task, line detection using the Hough Transform is described in the end of this session's guide, as well as a suggestion of a Simulink implementation.

## Objectives

- Familiarization with common edge, corner and circle detection in images
- Study the application spatial filtering on image segmentation

## MATLAB and Simulink functions

- Edge detection: `edge`, `fspecial`, `imfilter`, *Edge Detection*
- Sharpening filter: `imsharpen`, `imfilter`, `fspecial`, *2-D FIR Filter*
- Corner detection: `detectHarrisFeatures`, *Corner Detection*, *Draw Markers*
- Circle detection and plotting: `imfindcircles`, `viscircles`
- Line detection: `hough`, `houghpeaks`, `houghlines`, `plotLines` (provided MATLAB function), *Hough Transform*, *Find Local Maxima*, *Hough Lines*

## Tasks

In this fourth session, a MATLAB code implementation is suggested since it has functions for all the features we wish to identify in an image (edges, corners, lines and circles), and provides a simpler execution.

The first to be used in this session is a photography of a tire (*tire.tif*), which will be employed for edge detection, and the second presents a set of objects (*pillsetc.png*), useful for identifying corners and circles. Please note that the *tire.tif* is already a grayscale image, and therefore does not require previous conversion. These two images are shown together in figure 4.1.

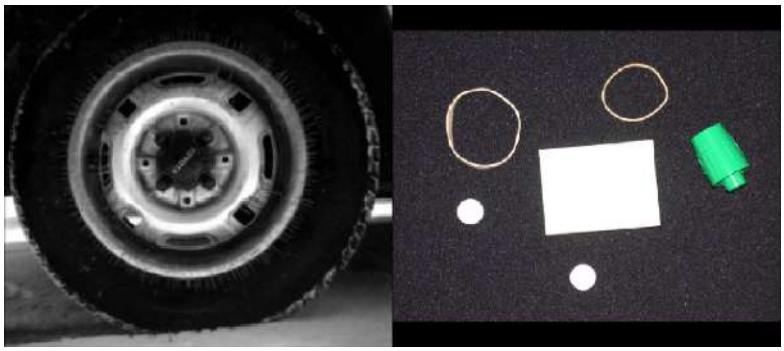

Figure 4.1: Image of *tire.tif* and *pillsetc.png* image (credited to MathWorks).

**1. Edge Detection and Sharpening Filters**

**Expected time of completion for task: 20 minutes**.

Firstly, in MATLAB, load the image of the tire shown in figure 4.1, and apply an **edge detection method** using an adequate MATLAB function. These can have several inputs and parameters, so be sure to read the documentation before. Try two or three different methods, evaluate their performance, then choose one and move on to the second part of this task. An example of edge detection using the Canny Edge Detector is shown in figure 4.2.

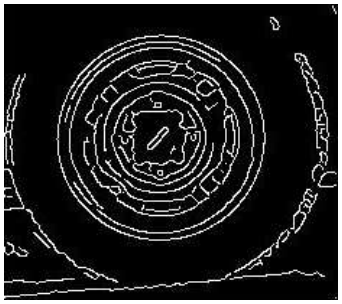

Figure 4.2: Edges *tire.tif* using the Canny Edge Detector.

As you can conclude from inspecting figure 4.2, the edge detection algorithm already provides good

results in showing the edges of the tire. However, it can be seen on the top-left section of the image that the edge is not segmented correctly, which is mainly due to the background being difficult to distinguish from the tire itself. If the objective is to detect the whole edge of the tire, this could be solved by an appropriate image processing technique (which one or ones do you think would be useful?), but, for the purpose of this laboratory session, you will use an **image sharpening filter** so that it sharpens the edges. Figure 4.3 illustrates an example of this, before and after implementing a sharpening filter prior to the edge detection.

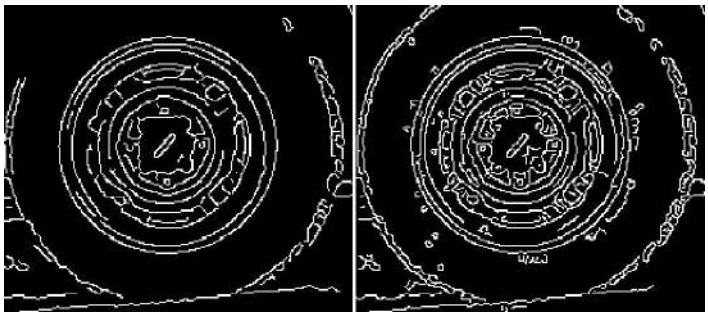

Figure 4.3: Edges *tire.tif* using the Canny Edge Detector before and after applying a sharpening filter.

There are several ways of achieving similar results to the ones shown on figure 4.3, identically to the smoothing filters on the second session, either by using specific functions for sharpening filters or by applying a mask with sharpening effects using a generic filter function. Can you think of another way of facilitating the detection of the edge of the tire?

**2. Corner Detection**

**Expected time of completion for task: 20 minutes**.

Now, load the second image, shown on the right of figure 4.1, and use a MATLAB **function for detecting corners** on your image. Figure 4.4 shows the result of the direct implementation of a corner-detecting function on the image of *pillsetc.png*.

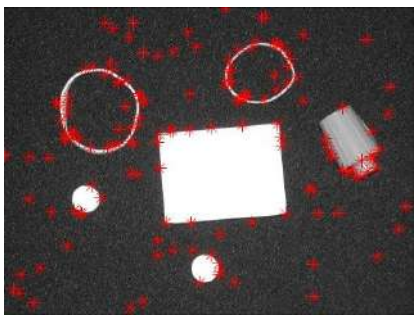

Figure 4.4: Corners of *pillsetc.png* using the Harris Corner Detector.

As you may agree, the results differ largely from it is expected since there are many more points of

the image that algorithm interprets as corners in the background. Why do you think this happens? Can you think of a method of solving this issue in order to detect only the corners of the white-rectangle and of the object at the right? A possible solution for this problem is shown below, in figure 4.5.

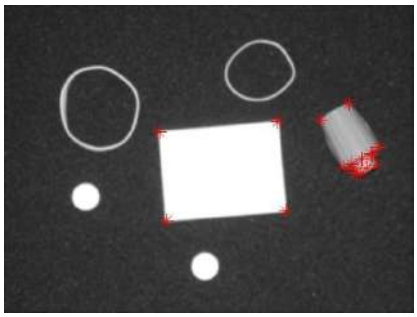

Figure 4.5: Corners of *pillsetc.png* after processing the image before the detection.

Take a note that some methods use a type of MATLAB variable that does not give the location of the corner points directly, and thus make it difficult to plot them like it is shown in figure 4.5. Try extracting the location of the points by inspecting such variable in the workspace, and then display them using the `plot` function.

**3. Circle Detection**

**Expected time of completion for task: 10 minutes**.

Lastly, use the same image of *pillsetc.png* and try to detect to detect circles this time. There are built-in MATLAB functions for this sole purpose which employ the Hough Transform and provide the centres and radii of the detected circles, so this task is supposed to be quite direct. Use a function for **circle detection with an interval of radii of 10 to 200** to the original grayscale image. Then, plot the detected circles on your original image using the ***vsicircles* function**, as shown in figure 4.6.

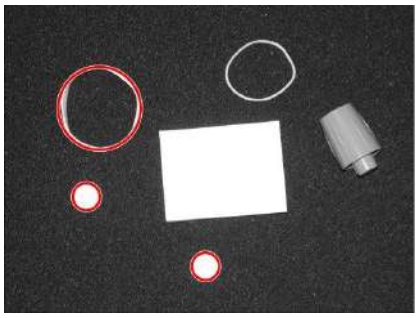

Figure 4.6: Circles detected in *pillsetc.png*.

Your results may differ from above, and you may actually manage to identify the last somewhat circular object. However, if you cannot, try to apply the same processing technique as in Task 2 and see if your results improve. If they do, can you think of a reason why?

**Further Work: Line detection and Simulink implementation**

As an advanced topic, this time the objective is to detect lines. Please, remember that **a line is not an edge**, and therefore the detection method requires a different tool. In this part, the lines of the binary image *blobs.png* can be detected using the following procedure: firstly, compute the Hough Transform of your image using MATLAB's `hough` function; after, obtain the maxima of your result employing the `houghpeaks` function (take care that the default number of peaks do detect is only one); lastly, obtain the start and end points of each line from the function `houghlines`, which takes as inputs the output variables of the previous two functions. Naturally, reading the `help` information for each of these functions is strongly recommended.

Now you should only have to plot the lines on your original image. A `for` cycle is suggested so you can plot all the lines sequentially, knowing their start and end points, but the supplied script `plotLines` can be used for this purpose, but it must be adapted into your code. If you set the number of peaks to be identified equal to 10, your result should be the following similar to the one shown in 4.7.

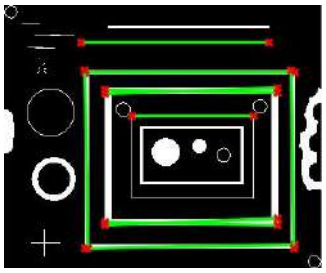

Figure 4.7: Lines detected in *blobs.png*.

- Simulink implementation

  As you may have seen, the MATLAB implementation for edges, corners and circles is quite simple since it provides built-in functions for detecting all the features in this laboratory session. However, Simulink does not offer functions for detecting circles, and the line detection using the required blocks is non-trivial. Therefore, we will focus on **edges** and **corners**, which can be obtained from a Simulink model like the one shown in figure 4.8.

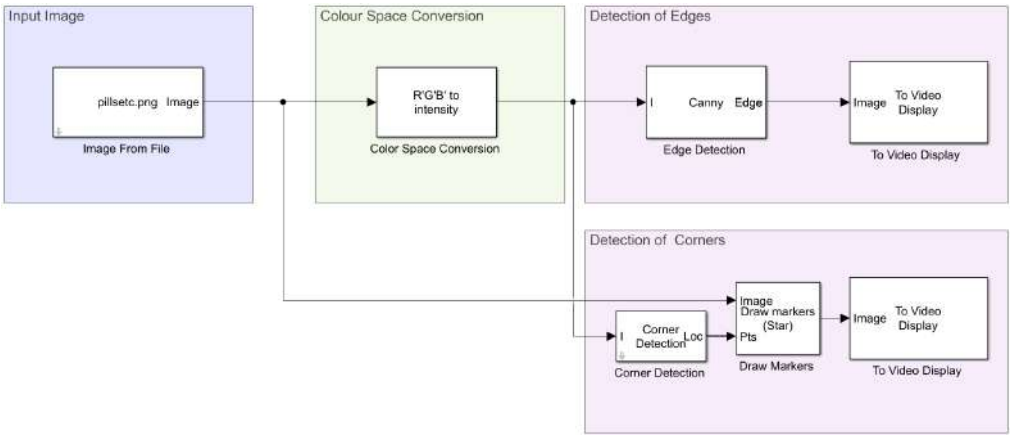


Figure 4.8: Suggestion of a Simulink model for edge and corner detection.

As for the edges, the *Edge Detection* block directly outputs a binary image of the edges it perceives, and provides several different methods, similarly to the analogous MATLAB function. On the other hand, the *Corner Detection* block only outputs the location of the corners, which must be then plotted with the original coloured image by using the *Draw Markers* block. Do not forget to **load your image in "*Double*" type** using the blocks *Image From Workspace* or *Image From File*, and to **set the sample time to a discrete value** for these blocks as well. The filtering, either for smoothing or for sharpening the image, can be achieved similarly to what was done in the second laboratory session, using the *2-D FIR Filter* block with an adequate mask for each of these operations.

Lastly, if you wish to detect lines, you can do so by using the three step method described for a MATLAB implementation with the blocks *Hough Transform*, *Find Local Maxima* and *Hough Lines*. However, complications may arise due to differing size images, which make the Simulink implementation complex and intricate, but, nonetheless, possible. If your objective is also to detect circles, a way of calling MATLAB functions in directly Simulink is advised: this can be achieved by using the `coder.extrinsic(function)` command on a *MATLAB Function* block, like it was described at the end of Chapter 2.

# Chapter 5

# Image Description Based on Keypoints

As the next topic for Part 1 of the course, an introduction to image description is made. Particularly, in this fifth class, the focus is to learn how to describe images using **keypoints**, also called **interest points**, which can then be gathered and used for feature extraction, forming **descriptors**. In turn, features detected in new images can be matched using such descriptors, allowing for image recognition, classification, and reconstruction, among other applications.

## Objectives

- Study the available feature detection methods
- Learn how to use feature extraction and matching for classification

## MATLAB and Simulink functions

- Feature detection: `detectSURFFeatures`, `detectHarrisFeatures`, `detectBRISKFeatures`, `detectFASTFeatures`, `detectORBFeatures`
- Feature extraction: `extractFeatures`
- Feature matching: `matchFeatures`
- Classification: `lab5_buildDescriptors` (provided MATLAB script), lab5_OnlineClassifier (provided Simulink model)

## Tasks

There are several methods to detect features in images, so this laboratory session will start by exploring some of these, in order to study how they work. Afterwards, a method of classifying objects is to be developed in Task 2, using a provided dataset of different geometrical objects, which will allow to understand how interest points can be used for feature matching. The further work for this laboratory session suggests the implementation of Task 2 in Simulink, as an introduction to online classification, and addresses the employment of feature matching to recover a distorted image, expanding the application of these methods beyond classification.

For the first task, any grayscale image can be used, since the aim is to compare the different feature detection methods. As an example image, this guide uses the *cameraman.jpg* image, shown next:

Figure 5.1: Example image for feature detection *cameraman.jpg* (credited to MathWorks).

For Task 2, images of different geometrical shapes - rhombuses, squares and triangles - are used, provided with the files for this laboratory session. In figure 5.2, an example of these three shapes are shown.

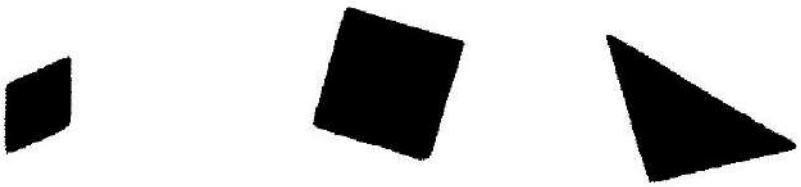

Figure 5.2: Geometrical shapes used for classification.

**1. Interest Point Detection**

**Expected time of completion for task: 10 minutes**.

As an introductory task, use the **MATLAB functions for feature detection** and try to draw some conclusions about how these work. There are several functions available for this end, but in this session we will focus on the SURF, BRISK, FAST and ORB features, but you can explore other ones as further work.

These MATLAB functions output a `points` object, which can be `SURFPoints`, `BRISKPoints`, `cornerPoints`, or `ORBPoints`, naturally dependant on your method of choice. Nonetheless, these objects have a multiple fields, which can be accessed by using a dot "." after the name of the object. For example, you can access the *Location* and *Count* fields of a `points` object for plotting the position of the interest points and to count them as well as illustrated in figure 5.3, using the following commands:

```
numberPoints = points.Count;
plot(points.Location(:,1), points.Location(:,2), 'r*')
```

Figure 5.3: SURF features detected and plotted in the *cameraman.tif* image.

Note that, depending on your image and the selected method, the number of interest points may vary greatly. It could be useful to select only a predefined number of points, so you can compare the detection methods with relative ease, which can be made using the `selectStrongest` attribute for your `points` object, which allows to consider only a specified strongest interest points among the ones detected.

Note that the expected time of completion of this task is 10 minutes, so experiment with two or three methods, try to limit the detected number of points, and then move on to the next task.

## 2. Offline Object Classification

**Expected time of completion for task: 40 minutes**.

For this second task, you should **use the provided files** - the `lab5_buildDescriptors.m` script and the "*datasetImages*" folder - since these will help build a set of descriptors you can use for classification of objects. The images from this folder show rhombuses, squares and triangles of different sizes and orientations, and are divided into two groups: the **building images**, which are the ones that end with "build" (*rho_build(3).jpg*, for example) as these will be used for generating the descriptor data, and the **testing images**, ending with "test" (*tri_test(8).jpg*, for example), which are naturally the ones that you should use to test your classifier.
The MATLAB script `lab5_buildDescriptors.m` works by callig a function that analyses the building images using a feature detection method (it uses the SURF algorithm, but this can be changed), and then generates a `struct` variables for each of these geometric shapes, containing the respective descriptors. In turn, these descriptors can be compared to the ones from another image - this time a test image - allowing to classify its geometric shape.
This task is somewhat complex, and thus should be divided in the following steps:

1. Firstly, run the provided script `lab5_buildDescriptors.m`, and understand how it **detects and extracts features**. After running this function, an image similar to the one in figure 5.4 should appear, showing the different interest points detected for each of the geometric shapes used for building. You should also note that three new variables - `descripRho`, `descripSqu` and `descripTri` - are now in your workspace, which include the features extracted for each class.

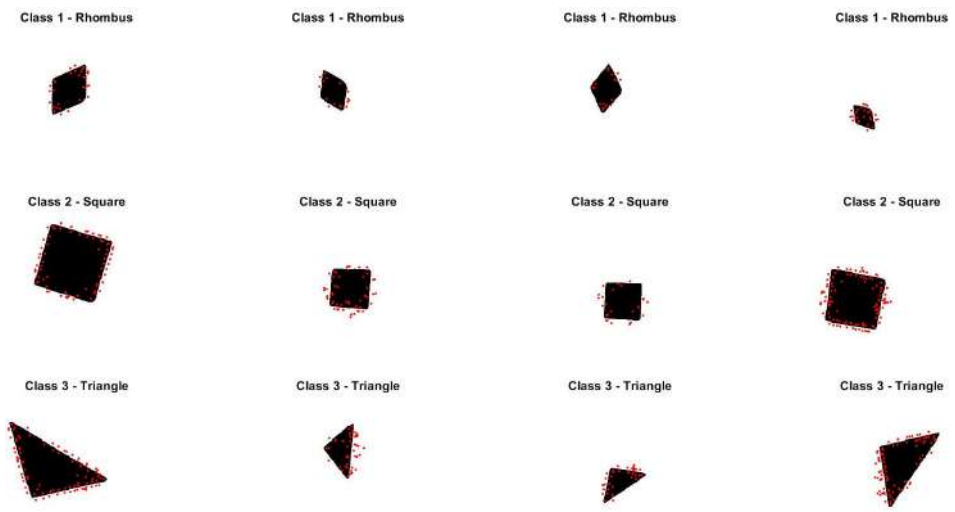


Figure 5.4: Interest points detected in the images used for generating the descriptors for each class.

2. Having generated a set of descriptors for each class, load one of the test images, and include the steps of detecting and extracting its features. As you may expect, if the goal is to compare such features to the ones in the descriptors, you should **use the same feature detection method**.

3. Now you can compare the features of your image to the ones of the descriptors by **finding a way of matching the features**. Remember that you can access the `features` field of any descriptor `struct` simply by using a dot: `descriptors.features`. Do this comparison for every class, and now you should have the number of matched features between the test image and the ones used for building the descriptors.

4. Lastly, **find a criteria to classify the geometric shape** of your test image. A good place to start is the number of matched pairs of features between your image and each class, but try to think about this topic and see if you can develop a better solution.

An example of this is shown in 5.5, which outputs the test image *tri_test(8).jpg* and plots the matched features for each class, and shows the number of matches with each class.

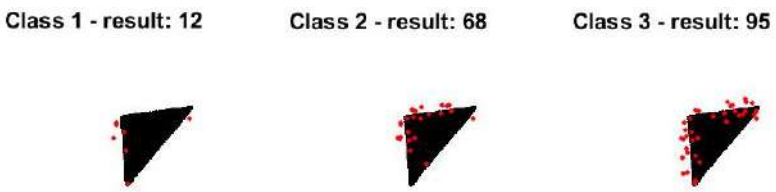


Figure 5.5: SURF features matched in *tri_test(8).jpg* for each class.

If you want to plot the matched points in the original image as shown in figure 5.5, you can include the two following lines of code, where `matchedPoints` result from your feature-matching step, and `points` is the variable that results from the feature detection:

```
points2Plot = points(matchedPoints(:,2))
plot(points2Plot.Location(:,1), points2Plot.Location(:,2), 'r*')
```

If you still have time, you can try to improve your classification process. This can be done in several ways, but two are suggested:

- <u>Feature matching</u>
  The function `matchFeatures` allows for additional parameters to be specified. A recommended parameter change is to set the `Metric` to `normxcorr`, since it will use a normalized cross-correlation matrix to analyse the features of your class and of your test image. On the other hand, setting `Unique` to `true` will eliminate repeated matches, so you might find it useful if you have unbalanced classes.

- <u>Classification criteria</u>
  Should the criteria be solely the maximum number of matched features? Or it should take into account the number of features of each class of your dataset? Think about this and try to improve your criteria so it works best.

**Further Work: Online classification and image recovery**

The first topic for further work is to **implement your classifier in Simulink** using the provided model, ***lab5_OnlineClassifier.slx***, which is shown in figure 5.6.

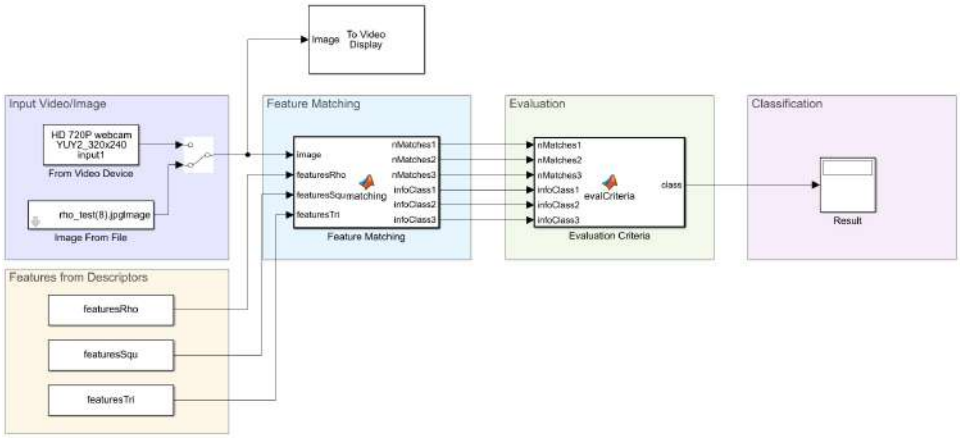


Figure 5.6: Provided Simulink model for online classification.

The Simulink blocks *Feature Matching* and *Evaluation Criteria* are function blocks where you can **implement your code from Task 2**. They already include several lines that enable a real-time adaptation of your code. The outputs of these function blocks are set to zero, but feel free to change so that they suit your method for classifying the geometric shapes. The two-block division suggests that you implement your feature-matching operations in the first and output the necessary information, and then the second block uses such information to classify the image according to your evaluation criteria.

If you manage to implement your offline classification algorithm successfully in Simulink, **you can try an online classification using the cameras of the computer**. The original geometrical shapes are available on request, so place them on the board below the camera (only one is needed), and then uncomment the *From Video Device* block and change the signal routing, so that geometric shapes are detected directly from the video feed.

For a successful implementation, account for the following aspects:

- The `lab5_buildDescriptors.m` script should be run before starting the simulation;
- The feature detection method that you implement inside the *Feature Matching* block should correspond to the one you used when calling the `lab5_buildDescriptors.m` script for generating the descriptors;
- The *Feature Matching* block uses the descriptors of each class as inputs and enables the different feature detection, extraction and matching functions, so only edit after this. Additionally, this function is commented with guidelines to help you.
- The image loading block and should provide grayscale images in 320 by 420 pixels, at a sample time of 0.1 seconds, and the same specifications should be kept if you work with the video block.

As continuation for Further Work, a different application of feature extraction and matching is suggested, now you have seen that image description using keypoints is a powerful tool, and that MATLAB allows to use it with relative simpleness. Nonetheless, the applications of feature extraction using keypoints go beyond classification, and thus are suitable for use in many other ends. Among one of these is **image recovery**, and, as further work, you should analyse the provided function `lab5_FurtherWork.m` to understand how an image can be recovered using keypoints.
The starting image is the *peppers.png*, which is then distorted using a two-dimensional projective transformation. These two images are shown in figure 5.7.

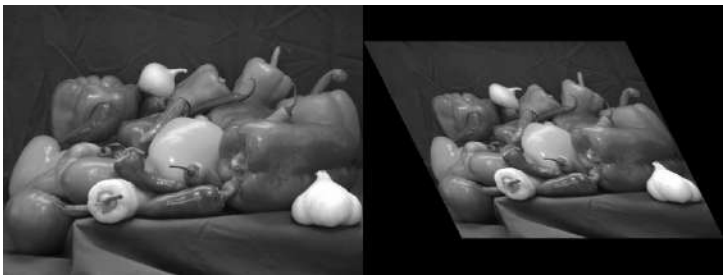

Figure 5.7: Original grayscale image of *peppers.png* (credited to MathWorks) and distorted version.

Afterwards, detecting and matching the features of both images allows to estimate the geometric transform, which in turn enables the recovery of the original image. Figure 5.8 illustrates the keypoint-matching process.

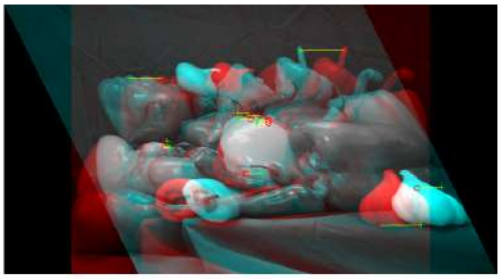

Figure 5.8: Matched keypoints between the original and distorted images.

Read the documentation of any function you do not understand, and try to replicate this procedure using other images, so you are aware of each step of the process.

# Chapter 6

# Image Description Based on Shape

The topic of this laboratory class is region-based image segmentation and description, aiming to complement the methods of image analysis using keypoints from the previous theoretical class and respective laboratory session. Two objectives are intended for this sixth session, the first being to obtain the region properties of a given binary image, and the second the employment of region-based methods for image description and classification. As further work, the employment of superpixels is suggested as a way of studying the application of this method to image segmentation.
This session also closes the first part of the laboratory classes, marking the end of the Image Processing component of this course.

## Objectives

- Familiarization with superpixels and how these can describe images
- Learn to extract region properties of binary images
- Explore and analyse online shape classification

## MATLAB and Simulink functions

- Extract region properties: `regionprops`
- Complement of a binary image: `imcomplement`
- Classification using keypoints and Hu Moments: `lab6_init`, `lab5_buildDescriptors`, `lab6_HuMomentDescriptors`, `HuInvariantMoments` (provided MATLAB functions/scripts), `lab6_OnlineClassifier` (provided Simulink model)
- Superpixels: `superpixels`

## Tasks

In this laboratory session, you will firstly extract region properties of binary images, which can be used for classification. Secondly, you will perform online classification using the Hu Invariant Moments, and then compare the classifier with the one designed on the previous laboratory session. For these two tasks, the images to be used are the ones of geometrical shapes - rhombuses, triangles and squares - in order to extract the relevant geometrical properties from them so they can be classified. Task 2 then uses these properties to implement the classification algorithm, the final goal being to perform live classification of such shapes using the cameras available. Another set of examples of these shapes is shown in figure 6.1.

Figure 6.1: Geometrical shapes used for classification.

### 1. Extraction of Region Properties

**Expected time of completion for task: 20 minutes**.

Now, load an image of the provided dataset of geometrical shapes, and try to **extract its properties** using adequate MATLAB functions. Note that some images, despite appearing binary, are not, and therefore you should **threshold these images first**. Skipping this step could lead to more detected regions than the one desired, compromising your analysis. It may also be useful to invert them using the `imcomplement` function, in order to **ensure that the white pixels are the ones that correspond to the geometrical shape**, since this convention is often adopted.
An example of the extraction of properties of a binary image is provided in figure 6.2.

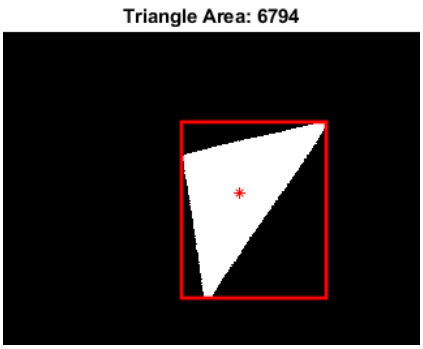


Figure 6.2: Binarized image of a triangle inside a bounding box, with its centroid and area( in pixels).

Read the documentation of the `regionprops` function, and think what other properties would be useful if you were to classify objects - geometric shapes, for example - with them. Also note that the geometric shapes may not be the single object detected in the images, and therefore this function will compute the properties of each region accordingly.

## 2. Object Classification

**Expected time of completion for task: 20 minutes**.

The last task of this session is to use the provided Simulink model *lab6_OnlineClassifier.slx*, shown in figure 6.3, to classify geometrical shapes. This model can work both offline, receiving and classifying the images that are loaded in the Simulink model, or online, providing live classification of the detected objects, by changing the switch at the left. The script `lab6_init` should be run before the Simulink model, as it creates the necessary variables.

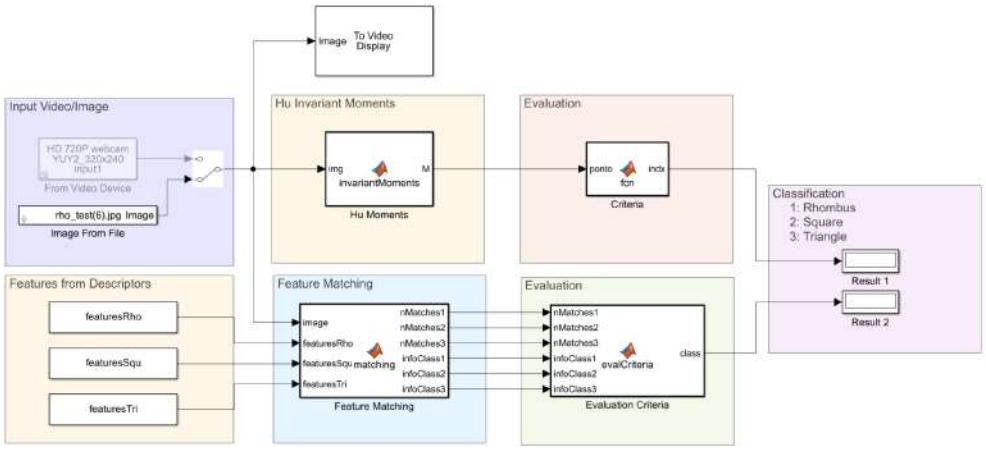


Figure 6.3: Simulink model for classification based on shape and based on interest points.

For this particular task, it is recommended that you do it using the computers and cameras of the laboratory because these have been previously configured, specially if you try to perform live classification using video. If you want to use your own computer, you should account for the following aspects:

- The image loading block and video block should provide **grayscale** images in **320 by 420 pixels**, at a **sample time of 1/30 seconds**, and with a ***Single*** **output data type**.
- You should have **installed the *Image Acquisition Toolbox Support Package for OS Generic Video Interface* add-on** (credited to MathWorks), and restart MATLAB afterwards.
- The files for the classifiers, which are described next, **should be on the working directory** of the Simulink model.

The model on figure 6.3 allows for two different classifiers to be used simultaneously and these are briefly explained now. The first classifier relies on the computation of the Hu Invariant Moments for ex-

tracting properties of each image, and then employs **Principal Component Analysis** (PCA) to obtain region-based descriptors. Afterwards, a given image can be analysed computing the respective Hu Moments and obtaining the centre of mass of the object from the Euclidean distance, using this last value as the basis for the classification. This classifier uses the provided `lab6_HuMomentDescriptors` and `HuInvariantMoments` functions, designed by Professor Rogério Caldas Pinto. The second classifier is the same of the previous laboratory class, based on the matching of SURF points, and uses the number of matched pairs as an evaluation criteria.
Following the aforementioned steps and understanding how both classifiers work, **try to classify the geometrical shapes**, firstly offline, and then online using the objects available at the laboratory. Strive to **assess the advantages and limitations of these two classifiers**, and think of ways to improve them.

#### Further Work: Segment an image using superpixels

For this task, you can work with any image, but it is recommended you do so with one that has distinguishable regions, for which superpixels can show their purpose. As an example, this guide shows the employment of superpixels to the *peppers.png* image, shown in figure 6.4.

Figure 6.4: Example image for superpixel application *peppers.png* (credited to MathWorks).

Firstly, load a coloured image, and then **use the `superpixels` function** with a given number of desired superpixels. Be sure to **extract the two outputs of the function**, since the first will provide you the superpixel labels, and the second will be the number of available labels (which will actually be the number of superpixels). Using the provided functions and the obtained labels, it is now possible to obtain an image that shows how the superpixels segmented the image, which is shown in figure 6.5.

Figure 6.5: Superpixel segmentation on *peppers.png*.

Try using different values for the desired number of superpixels of your image, and watch how it affects the overall segmentation.

This shows how the image becomes ”separated” into different regions but, as you may see from figure 6.5, the image itself still has as many pixels as the original. Therefore, the next step is to ensure that every pixel inside each region has the same red, green and blue values. This can be achieved using the `mean` function, and a possible implementation is shown in Algorithm 2, in which `Labels` and `nLabels` present the outputs of the `superpixels` function.

**Algorithm 2** Generating a superpixel RGB image

```
outputImg=zeros(size(inputImg),'like',inputImg)          ▷ Initializes the output image
indexes=label2idx(Labels)                                ▷ Extracts indexes from labels
numRows=size(inpImg,1)
numCols=size(inpImg,2)
for labelVal=1,...,nLabels do
redIdx=indexes{nLabel}
greenIdx=indexes{nLabel}+numRows*numCols;
blueIdx=indexes{nLabel}+2*numRows*numCols;
                    ▷ Obtain the indexes for the red, green and blue for each superpixel region
outputImg(indexR)=mean(RGB(redIdx));
outputImg(indexG)=mean(RGB(greenIdx));
outputImg(indexB)=mean(RGB(blueIdx));
                    ▷ Obtain the mean of each red, green and blue value for each superpixel region
end for
```

Following this method, the final image should be composed of `nLabels` regions. For each region, all of the correspondent pixels will have the same values for their red, green and blue components, like the image shown on figure 6.6, which results from the application of Algorithm 2 to figure 6.5. Note that your results may vary significantly, since the obtained superpixels are a consequence of your original image and the desired number of superpixels you provide.

Figure 6.6: Superpixel image after segmentation of *peppers.png*.

# Part II

# Computer Vision

# Chapter 7

# Intelligent Vision Systems

Introducing the second part of the laboratory sessions, the goal is to study how intelligent vision systems can be used to extract information from images, namely how to employ these tools in classification problems.
For this laboratory session, two different models for classification based on intelligent systems will be studied, which rely on image properties to classify them as belonging to a certain class.

## Objectives

- Study and apply different classification models
- Learn to classify images based on classification models
- Analyse the obtained results using adequate metrics

## MATLAB and Simulink functions

- Extract region properties: `regionprops`, `lab7_main` (provided MATLAB script), `lab7_BuildTable` (provided MATLAB function)
- Partition data: `cvpartition`
- Build models: `fitcknn`, `fitcnet`
- Predict result: `predict`
- Obtain confusion matrix: `confusionchart`

## Tasks

As it was done in the two previous laboratory sessions, you will work with the sample images of geometrical shapes and with an additional MATLAB script `lab7_MAIN`, which is provided in the files for this session, together with the dataset of images, and calls the other provided function `lab7_buildTable`. This function processes the images so the geometrical shapes are represented by white pixels as shown in figure 7.1, calling the `imcomplement`, `imbinarize` and `imerode` functions, extracting the respective properties by using the `regionprops` function.

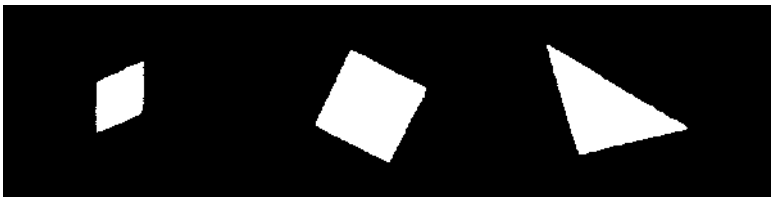

Figure 7.1: Images used to extract properties of geometrical shapes.

The extracted properties are then grouped in a table for convenience, together with the class of each image, and stored in the `propsTable` variable. By default, this first step generates a table with only 2 fields - **Area** and **Perimeter** - being properties of each geometrical shape. The last column corresponds to the **Class** to which the geometrical shape of each image belongs to. This table is shown in figure 7.2.

| Area | Perimeter | Class |
|---|---|---|
| 3862 | 263.27 | "Rhombus" |
| 2436 | 201.22 | "Rhombus" |
| 2250 | 190.56 | "Rhombus" |
| 1676 | 165.67 | "Rhombus" |
| 1522 | 156.26 | "Rhombus" |
| 1211 | 141.62 | "Rhombus" |
| 1105 | 133.36 | "Rhombus" |
| 1102 | 135.36 | "Rhombus" |
| 2675 | 209.18 | "Rhombus" |
| 11147 | 412.82 | "Square" |
| 13060 | 456.35 | "Square" |
| 11005 | 421.82 | "Square" |
| 4690 | 275.07 | "Square" |
| 4195 | 261.44 | "Square" |
| 3171 | 224.77 | "Square" |
| 8158 | 367.99 | "Square" |
| 8965 | 382.06 | "Square" |
| 9397 | 385.44 | "Square" |
| 10388 | 498.8 | "Triangle" |
| 3564 | 284.23 | "Triangle" |
| 7631 | 415.8 | "Triangle" |
| 2601 | 243.05 | "Triangle" |
| 2180 | 220.99 | "Triangle" |
| 4151 | 305.38 | "Triangle" |
| 4967 | 341.18 | "Triangle" |
| 5885 | 372.33 | "Triangle" |
| 6429 | 388.8 | "Triangle" |

Figure 7.2: Table of properties of the images of geometrical shapes.

The function then partitions the data of the table into a **training set** and a **testing set** using the `cvpartition` function, being randomized subsets of the original data. By default, the testing set is made to be 20% of the original data. The training data is represented by the variables `XTrain`, which are the properties, and `YTrain`, which are the correspondent classes. Analogously, `XTest` and `YTest` represent the testing data.

## 1. Classification using a *k*-Nearest Neighbour Model

**Expected time of completion for task: 20 minutes**.

The first task is to generate a *k*-Nearest Neighbour (KNN) model that fits the training data, and then evaluate how well this model predicts the classes for the testing data. For this goal, after running the `lab7_MAIN` script, start by creating a KNN model that uses the `XTrain` and `YTrain` variables. Afterwards, use the `predict` function to obtain the classes for the `XTest` data, and **compare** your predictions with the `YTest`. Can your model successfully predict the class of all the test images?
For a more systematic analysis, **compute the accuracy** of your model. Computing the accuracy of the predictions can be done using (7.1).

$$Accuracy = \frac{number\ of\ correct\ predictions}{total\ number\ of\ predictions} \times 100 \tag{7.1}$$

It is useful to use the `strcmp` (string compare) function for comparing your predictions with the correct classes. It will output a vector of ones and zeros, corresponding to correct and incorrect predictions, respectively. From this vector, you can compute the number of correct predictions by computing its sum, and obtain the total number of predictions using the `length` function.
Lastly, **obtain the confusion matrix** from your results using the `confusionchart` function. An example is shown in figure 7.3, which corresponds to a model prediction accuracy of 80%.

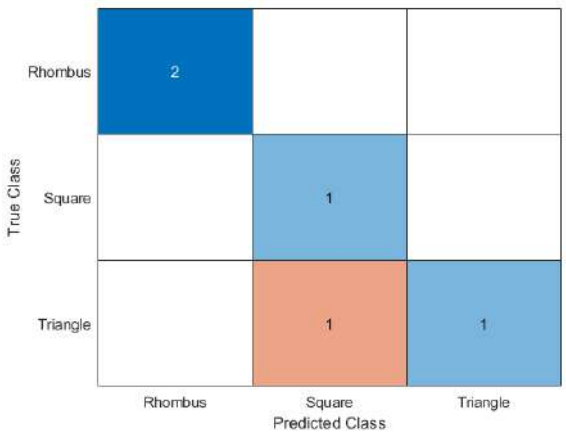


Figure 7.3: Confusion matrix.

The provided dataset is quite small, and therefore your results may vary significantly according to the data partition, so make sure you run your model several times in order to get an idea about its performance. If you have time, feel free to use the information provided in the `help` command for the KNN model to improve your classification model.

**2. Feature Influence in Classification Results**

**Expected time of completion for task: 20 minutes**.

Now that you have a working model, think about the properties, or **features**, being used to build your KNN classifier and make predictions. Is the area a relevant feature to distinguish a triangle from a square or a rhombus? Or should you look into the circularity of the shape?
To understand how each feature influences the classification, visualizing how each pair of properties are distributed among the classes may be of help. In the provided script, the initial lines of the code allow you to change properties and see how these are distributed for each class. An example of this process using the Perimeter and Area properties is shown in figure 7.4.

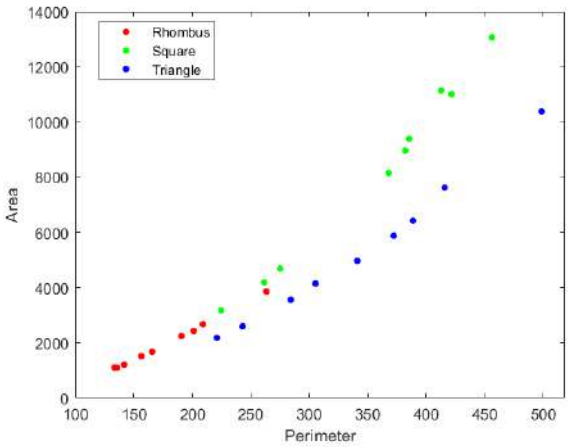


Figure 7.4: Perimeter and area values for each class.

Try to think about the features that make most sense to you from the ones available using the `regionprops` function, and then change the `properties` vector. Note that it is a string array, so additional entries must be formatted accordingly. Afterwards, evaluate the performance of your model when selecting different features, and try to get the best and most-consistent results. What is the combination that works best for you? Is there a particular feature or group of features that you consider as essential in this classification? Can you think of a reason why that happens?

**3. Classification using Neural Networks**

**Expected time of completion for task: 10 minutes**.

The last task is to repeat the last steps, but using a neural network as a classifier instead. **Build a neural network classifier** with the `fitcnet` function and experiment with different number of layers, layer sizes and activation functions, or other additional parameters you consider suitable, and see how these impact the results. Is there any way to make your model get an accuracy of 100%?

# Chapter 8

# Deep Learning for Image Recognition

Having already studied some machine-learning applications for image classification using specified **features** of the objects that were supposed to be identified, you will now get the chance of using **Deep Learning** to perform similar tasks. For this laboratory session, you will study how deep neural networks can be used for image classification.

## Objectives

- Use a pre-trained neural network for generic image classification
- Create a simple neural network for digit classification
- Analyse the obtained results

## MATLAB and Simulink functions

- Load GoogLeNet pre-trained net: `googlenet`, `lab8_task1_PretrainedNet` (provided MATLAB script), `lab8_task2_DigitClassification` (provided MATLAB script)
- Resize image: `imresize`
- Classify images: `classify`
- Training options: `trainingOptions`
- Train neural network: `trainNetwork`

## Tasks

In this laboratory session, you will focus on deep neural networks for image classification. For this end, two approaches can be used, differing on whether a new network is to be designed or not. If the approach is not to design a new network, a pre-trained one must be used instead.
As a first task, you will use the GoogLeNet Convolutional Neural Network (CNN), which is a pre-trained neural network with 1000 classes. **Be sure to install this network before the laboratory session** using the Add-On Installer and searching for "Deep Learning Toolbox Model for GoogLeNet Network". As for the second task, a new neural network will be created, and then trained using a digits images dataset (credited to MATLAB), which is a familiar problem of Deep Learning applied to image classification.

### 1. Generic Image Classification using a Pre-Trained Neural Network

**Expected time for completing task: 20 minutes**.

The first task is to use a generic image and see how GoogLeNet classifies it. If you have not yet installed GoogLeNet, please do so now. A set of instructions is given in the `lab8_task1_PretrainedNet` script to help you in the implementation of this task, which is provided with the files for this laboratory.
Pre-trained networks for image classification have certain dimensions, namely for their input and output layers, since the input size must agree with the size of the images the network receives, and the output will correspond to the different classes or labels. In MATLAB, you can inspect the layers of neural networks by accessing its `Layers` property using `net.Layers` (in which "`net`" is the name of your loaded neural network). The first step in this task is to know the dimensions of the images GoogLeNet accepts, and how many labels it can classify images to. For this, **inspect the `InputSize` property of the first layer**, and **the `Classes` field of the last**. Can you think of adequate justifications for your results?
After you understand how GoogLeNet works, download a coloured image of a generic object from the Internet and resize it according to the `InputSize` property of the first layer of GoogLeNet using `imresize`. Try to **classify it** using the `classify` function and see if the result is adequate for the object in your image. An example of this using a generic image of a desktop computer is shown in figure 8.1, in which the classification obtained from GoogLeNet is provided in the title.

Figure 8.1: Classification of a desktop computer image from GoogLeNet (available here).

Despite pointing to a single class, GoogLeNet provides confidence levels for all the classes of the neural network, in which the result of the prediction is the one with the highest confidence level. Nonetheless, it is useful to evaluate how much confidence there is in the result, as well as the other classes of relevant confidence.

For the second part of this task, **extract both the labels and scores output** provided by the `classify` function, in which the first is the prediction and the second an array of prediction scores for each classes. If it is convenient for you, call them "pred" and "scores", which is also how the `lab8_task1_PretrainedNet` defines these outputs. Now, create a logical array which is one if the score is above a certain value (start by 0.01, for example), and zero when it is not. Afterwards, **use this logical array as an index for the Classes** (which can be obtained from the last layer of the neural network) **and for the scores**, which will allow you to know which are the classes that have a score above the threshold, and which scores they have. If you call these new variables "BestClasses" and "BestScores", `lab8_task1_PretrainedNet` will create a chart of vertical bars of the scores higher than the threshold, and show the correspondent classes, like shown in figure 8.2.

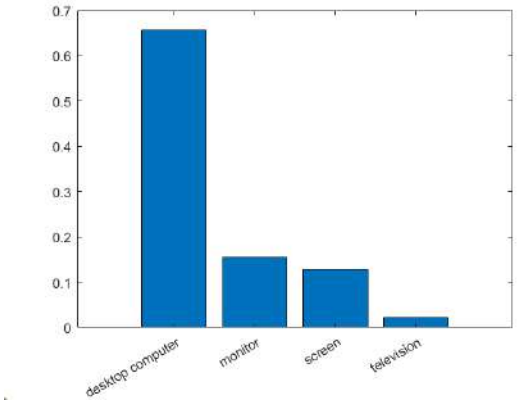


Figure 8.2: Classes with a score higher than 0.01 for figure 8.1 classification using GoogLeNet.

**2. Simple Deep Neural Network for Digit Classification**

**Expected time of completion for task: 25 minutes**.

After understanding a pre-trained network, the goal now is to build a neural network from zero, provided you have three things: a labelled dataset, a defined network architecture, and a set of training options. Each of these is described next, so you can design, train and implement this classifier, and, similarly to Task 1, the `lab8_task2_DigitClassification` script is provided since it contains some guidelines for this task.

- Datastore

  The required dataset of images is provided with the files for this laboratory on the folder "Digit-Dataset". It comprises 1000 images for each digit, from 0 (zero) to 9. An example of 12 random images of this dataset is shown in figure 8.3.

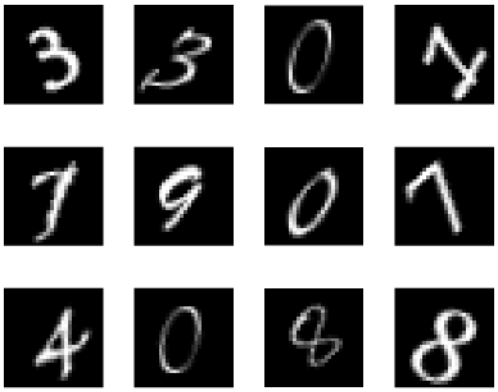

Figure 8.3: Example of 12 random images of the ”DigitDataset” (credited to MATLAB).

An **image datastore** is a tool that MATLAB offers for conveniently grouping and managing large sets of images, which is particularly useful for training and testing classification algorithms. The first part of the `lab8_task2_DigitClassification` script uses this method for creating a datastore of images from the dataset, and already splits it randomly into two groups, one for training (`dataTrain`), and one for testing (`dataTest`).

- Network architecture

  The architecture of the neural network is the baseline of how it works, since it includes each of its layers and how these are connected, thus representing the relation between its inputs and outputs. You should already have an idea about the purpose of the first and last layers from the previous task. The layers in between these can be of multiple types - convolutional, activation or pooling layers, among others. You can find detailed information about these and how they can be implemented in MATLAB in this link.

  For this task, **you will work with a pre-defined architecture**, which is the one provided in the `lab8_task2_DigitClassification` script. This provides a relatively simple neural network of 15 layers, but perfectly suitable for the digit classification goal.

- Training options

  Defining the training options is the last step before you can train your network. The `trainingOptions` function only requires one input, which is the solver, but it has multiple possible additional parameters and options. Read the information provided by the `help` command about this function, and think about which additional parameters could be useful. For your network, specify the training options for a ”Stochastic Gradient Descent with Momentum” solver with an **initial learning rate of 0.1**, and **set the maximum number of epochs to 4**. This should ensure a relatively fast training process, which is important in case your computer does not have a dedicated Graphics Processing Unit (GPU). You can use additional options that allow you to verify the progress of the training like `Verbose` and `Plots` options, but using these can slow the training process.

Once you have gathered these 3 elements, **use the `trainNetwork` function to train your network**. It may take from some seconds to a few minutes depending on your system, namely if it has a dedicated GPU or not.

With your trained network, **classify the images of your test dataset** and **compare them to the corresponding labels**, like it was done for the first task of the previous laboratory session. Figure 8.4 illustrates this, and the provided code should replicate this image as long as you name the result of the prediction made by the neural network "`YPred`".

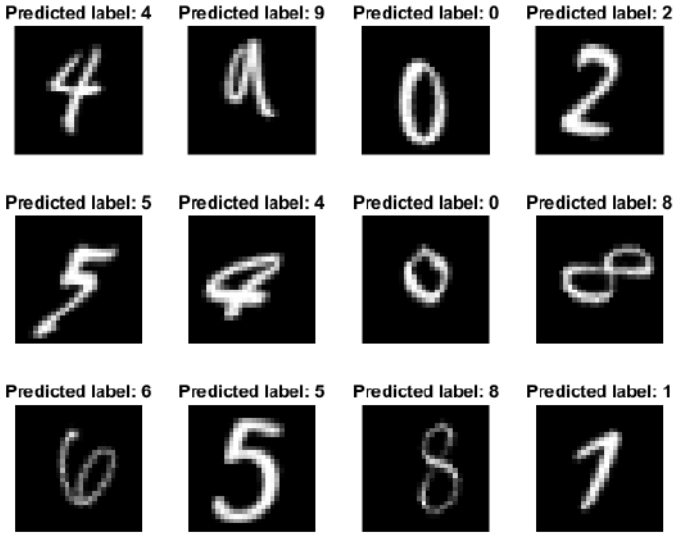


Figure 8.4: Classification of 12 test images of the "DigitDataset".

Lastly for this task, **obtain the accuracy and confusion matrix** for all the test data, like shown in figure 8.5, and **analyse the results**. Can you think of a reason for some of the mismatches?

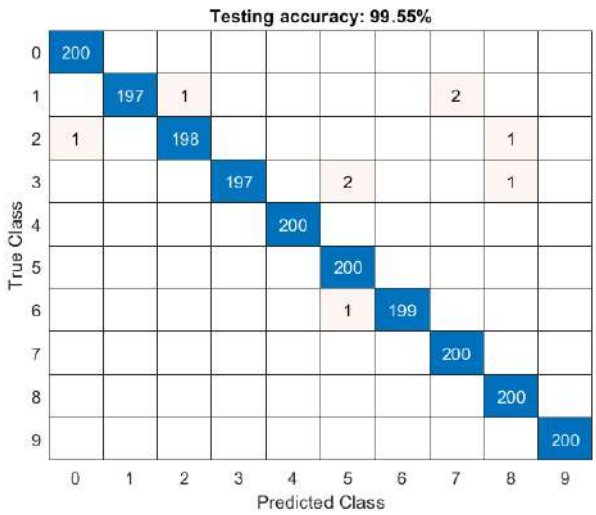


Figure 8.5: Confusion matrix for the test images, corresponding to a 99.55% accuracy.

**Further Work: Classification of Geometrical Shapes**

If you successfully completed the previous tasks, you can try to design a neural network classifier to classify the three geometrical shapes of the previous laboratory sessions. This time, the dataset is provided as a folder named ”images” with three subfolders, each comprising around 30 images. Each subfolder is named after the correspondent classes so you can use the ”LabelSource” option of the `imageDatastore` function to get the labels directly.

The provided script for further work, `lab8_FurtherWork`, loads the images of the geometrical shapes as a *datastore* to be used in the training process of a classifier, and uses the network from Task 2, with suitable adaptations to the input and classification layers. You can start by training this network with the generated datastore, and evaluate the obtained performance similarly to what was done in Task 2. Nevertheless, **think of this further work as a broad assignment**, with the general aim to improve the classification results: you can choose to **change the network architecture**, or even use a different and/or more complex one, such as the one from Task 1 (with the necessary alterations, naturally); you can **work on getting more images by pre-processing the ones you have**; or you can **think about in the training options** and try to get the best of what you have regarding the data and network architecture. Just be cautious of the computational power each of these components require. If needed, a program interruption (pressing ”CTRL” and ”C” simultaneously) can be used at every step of the training process.

# Chapter 9

# Camera Calibration

The last topic for the laboratory classes is the introduction to Stereo Vision, exploring the different camera models and parameters, as well as camera calibration. In this laboratory session, you will learn how to properly calibrate a setup of two cameras and to extract some relevant information using it.
Ensure that you can successfully connect to a USB camera and that you can access it from MATLAB/Simulink. It is **strongly recommended** you do this in the beginning of the class, as it will be used not only for the two remaining laboratory sessions, but also for certain topics of Project 2 of the course. Furthermore, it is useful that you bring small dark coins and a ruler for the second task.

## Objectives

- Study the mechanics of calibrating a stereo camera setup
- Comprehend how to analyse the image using the intrinsic and extrinsic parameters

## MATLAB and Simulink functions

- Estimate camera parameters: `estimateCameraParameters`, Stereo Camera Calibrator App, Camera Calibrator App
- Detect checkerboard: `detectCheckerboardPoints`
- Generate checkerboard corners: `generateCheckerboardPoints`
- Display extrinsic parameters: `showExtrinsics`
- Display projection errors: `showReprojectionErrors`
- Remove camera distortion: `undistortImage`
- Compute camera extrinsic parameters: `extrinsics`
- Convert image point coordinates to world coordinates: `pointsToWorld`

## Tasks

Two tasks are proposed for this laboratory session, the first being the **calibration of a stereo camera rig**, and the second the **measurement of objects in an image using a single camera**. These tasks represent initial topics of Stereo Vision, and therefore you will be following these MATLAB tutorials:

- Task 1: Using the Stereo Camera Calibrator App (tutorial video here)
- Task 2: Measuring Planar Objects with a Calibrated Camera

There are additional links you can explore if you wish to understand more about the underlying MATLAB functions, namely Camera Calibration, Using the Single Camera Calibrator App and Evaluating the Accuracy of Single Camera Calibration. Try to familiarize yourself with these topics, as they will be crucial for the last laboratory session, as well as for some of the topics of the second project of this course.
By following the instructions of the provided links, you should perform the two tasks without any issues. Nonetheless, this guide provides you with additional insight over the implementation.

### 1. Stereo Camera Calibration

**Expected time for completion of task: 20 minutes**.

The first part of this task requires a set of images taken from two cameras of the checkerboard pattern. These can be used directly from MATLAB (which have been provided in the files for this laboratory session), as shown in the recommended tutorial above, or you can **use the provided Simulink file and take images of the checkerboard pattern**. For the second option, ensure that: 1) the images from the left camera are correctly distinguishable from the ones of the right; 2) images from both cameras capture the pattern in full; 3) that you vary both the angle and the distance of the checkerboard to the cameras; and 4) most importantly, you **do not change the position and orientation of the cameras when taking pictures**. Afterwards, you can obtain the camera parameters, both intrinsic and extrinsic, either by using the Stereo Camera Calibrator App or programmatically by using the `detectCheckerboardPoints`, `generateCheckerboardPoints` and `estimateCameraParameters` and functions. Additionally, you can inspect the parameters calling `showExtrinsics`, as shown in figure 9.1, and to visualize the calibration errors computed using `showReprojectionErrors`.

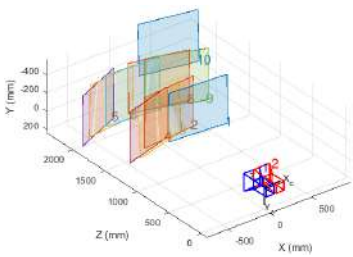


Figure 9.1: Visualization of extrinsic parameters after a stereo rig calibration.

**2. Measure Planar Objects with a Single Camera**

**Expected time for completion of task: 30 minutes**.

For this second task, you can once again take another set of images using the Simulink file, or you can use the ones shown on the tutorial for this second task, Measuring Planar Objects with a Calibrated Camera (which have been provided with the files). Naturally, the preferred option is to **take and use images taken by you**, ensuring that these are **images that contain both the checkerboard and an additional object to be measured**, which must be in the same plane as the pattern (a coin is suggested). However, you will use the images from only one camera, so, comment the second camera subsystem when taking your images.

Once you have the set of images for calibration, select one for measuring the object, and follow the aforementioned tutorial. The tutorial is a detailed guide, and it displays images for each step, so make sure you implement them step-by-step while fully understanding the idea behind each. It can be summarized to 5 steps after you have loaded a set of images which are, which are also shown in the provided `Lab9_task2` script:

1. Perform a single camera calibration using `detectCheckerboardPoints`, `generateCheckerboardPoints` and `estimateCameraParameters` .
2. Remove the distortion of the image to perform the measurement calling the `undistortImage` function.
3. Segment the coins by thresholding the saturation channel of the HSV image (for coin detection) and then detect them using `vision.BlobAnalysis`.
4. Compute the extrinsic parameters of the image in which to measure the object with the `estimateExtrinsics` function.
5. By calling `pointsToWorld`, convert the image coordinates of the coin to its world coordinates, and compare the measured value with its true value.

Additionally, you can obtain the distance from the camera to the coin as described in the tutorial, so try it if you have the time. The tutorial to be followed is extensive and you may be unable to finish this second task, but keep in mind that the key aspect to retain from this laboratory session is the camera calibration.